

Tarea – Cubos Multidimensionales

Table of contents

1	Introducción	2
2	Selección del conjunto de datos	2
3	Flujo de trabajo	2
4	Requisitos previos	3
4.1	Dependencias	3
4.2	Estructura local del proyecto	3
5	Ejercicio 1. Descarga automatizada desde Copernicus CDS	3
5.1	Script de descarga	4
5.2	Resultado esperado	5
6	Ejercicio 2. Procesamiento del cubo NetCDF	5
6.1	Lógica general del procesamiento	5
6.2	Comentario metodológico	6
7	Verificación del cubo procesado	7
8	Resultados gráficos	8
8.1	Figura 1. Mapa del máximo mensual	8
8.2	Figura 2. Serie diaria promedio del área de estudio	8
9	Resultados principales	8
10	Respuestas analíticas	9
10.1	Sobre evaluación perezosa	9
10.2	Sobre convenciones CF y tiempo	9
10.3	Discusión técnica	9

11 Conclusión	10
12 Anexo. Nota sobre el repositorio	10

1 Introducción

En esta actividad se desarrolla un flujo de trabajo completo sobre un cubo NetCDF de reanálisis climático. El proceso parte de la adquisición automatizada de datos desde Copernicus CDS y continúa con la lectura perezosa del archivo, la auditoría de dimensiones y metadatos, la transformación de unidades, el remuestreo temporal, el cálculo del máximo mensual por píxel junto con la fecha de ocurrencia y la exportación del resultado final.

La solución se implementó en Python con `xarray`, `rioxarray` y `matplotlib`. El conjunto de datos seleccionado fue ERA5-Land y la variable analizada fue la temperatura del aire a 2 m para un recorte espacial sobre Bogotá durante enero de 2020.

Objetivo del ejercicio

Construir un flujo de trabajo reproducible de procesamiento de un cubo NetCDF climático, manteniendo control sobre la memoria, la metadata y la dimensión temporal, y generando un producto derivado listo para análisis posteriores.

2 Selección del conjunto de datos

Se utilizó el dataset ERA5-Land con la variable `2m_temperature`, para enero de 2020, con resolución horaria y una subregión espacial recortada sobre Bogotá. Esta elección reduce el volumen del archivo descargado y facilita el procesamiento local sin perder el carácter multidimensional del cubo.

3 Flujo de trabajo

1. Registro en Copernicus CDS y configuración de credenciales.
2. Descarga del archivo NetCDF mediante `cdsapi`.
3. Lectura perezosa del cubo con `xarray`.
4. Auditoría de dimensiones, coordenadas y variable principal.
5. Normalización de nombres temporales y espaciales.
6. Conversión de Kelvin a Celsius.
7. Remuestreo temporal de horario a diario.
8. Cálculo del máximo mensual por píxel.

9. Identificación de la fecha de ocurrencia del máximo.
10. Generación de figuras de verificación.
11. Exportación del producto final a NetCDF.

4 Requisitos previos

El flujo de trabajo asume un entorno con Python 3.8 o superior, una cuenta activa en Copernicus CDS, el archivo `~/.cdsapirc` ya configurado con credenciales válidas y una carpeta local `data_heavy/` para almacenar entradas y salidas.

4.1 Dependencias

```
pip install cdsapi xarray rioxarray numpy pandas matplotlib netCDF4 dask cfgrib
```

4.2 Estructura local del proyecto

```
20_cubos_multidimensionales/  
  data_heavy/  
    era5_land_bogota_2020_01.nc  
    mi_zona_procesada.nc  
    temperatura_maxima_enero_2020.png  
  notebooks/  
  scripts/  
    descarga_cds.py  
    procesar_cubo.py  
  respuestas.qmd  
  README.md
```

5 Ejercicio 1. Descarga automatizada desde Copernicus CDS

La descarga se realizó mediante `cdsapi`, usando un request parametrizado con la variable, el periodo y el recorte espacial de interés.

5.1 Script de descarga

```
#!/usr/bin/env python3

import sys
from pathlib import Path

try:
    import cdsapi
except ImportError:
    print(" ERROR: cdsapi no está instalado")
    print("Instala con: pip install cdsapi")
    sys.exit(1)

data_dir = Path("./data_heavy")
data_dir.mkdir(exist_ok=True)

cds_config = Path.home() / ".cdsapirc"
if not cds_config.exists():
    print(" ERROR: No se encontró ~/.cdsapirc")
    sys.exit(1)

client = cdsapi.Client()

dataset = "reanalysis-era5-land"
target = data_dir / "era5_land_bogota_2020_01.nc"

request = {
    "variable": ["2m_temperature"],
    "year": ["2020"],
    "month": ["01"],
    "day": [str(d).zfill(2) for d in range(1, 32)],
    "time": [f"{h:02d}:00" for h in range(24)],
    "data_format": "netcdf",
    "download_format": "unarchived",
    "area": [5.2, -74.6, 4.2, -73.6]
}

client.retrieve(dataset, request, str(target))
print(f" Descarga completada: {target}")
```

5.2 Resultado esperado

El producto de esta etapa es el archivo `data_heavy/era5_land_bogota_2020_01.nc`, que constituye la entrada del pipeline de procesamiento.

6 Ejercicio 2. Procesamiento del cubo NetCDF

El script `scripts/procesar_cubo.py` implementa la lectura, la transformación de unidades, el remuestreo diario, el cálculo del máximo mensual y la generación del archivo de salida.

6.1 Lógica general del procesamiento

```
import numpy as np
import pandas as pd
import xarray as xr
import rioxarray
from pathlib import Path

project_root = Path(__file__).resolve().parent.parent
data_dir = project_root / "data_heavy"
ncfile = data_dir / "era5_land_bogota_2020_01.nc"
outfile = data_dir / "mi_zona_procesada.nc"

# 1. Lectura perezosa
ds = xr.open_dataset(ncfile, chunks="auto", engine="netcdf4")

# 2. Normalizar nombres
rename_dict = {}
if "valid_time" in ds.coords or "valid_time" in ds.dims:
    rename_dict["valid_time"] = "time"
elif "forecast_time" in ds.coords or "forecast_time" in ds.dims:
    rename_dict["forecast_time"] = "time"
if "lon" in ds.coords or "lon" in ds.dims:
    rename_dict["lon"] = "longitude"
if "lat" in ds.coords or "lat" in ds.dims:
    rename_dict["lat"] = "latitude"

if rename_dict:
    ds = ds.rename(rename_dict)
```

```

# 3. Selección y CRS
var_name = list(ds.data_vars)[0]
da = ds[var_name]

da = da.rio.set_spatial_dims(x_dim="longitude", y_dim="latitude", inplace=False)
da = da.rio.write_crs("EPSG:4326", inplace=False)

# 4. Transformación de unidades
da_c = da - 273.15
da_c.attrs["units"] = "°C"
da_c.attrs["long_name"] = "Temperatura del aire a 2 m en Celsius"

# 5. Remuestreo temporal
da_daily = da_c.resample(time="1D").mean()

# 6. Máximo mensual
da_monthly_max = da_daily.resample(time="1MS").max()
da_monthly_max.name = "t2m_max_mensual_c"

# 7. Fecha de ocurrencia del máximo
def fecha_del_maximo(grupo):
    idx = grupo.argmax(dim="time", skipna=True)
    return grupo["time"].isel(time=idx)

da_fecha_max = da_daily.resample(time="1MS").map(fecha_del_maximo)
da_fecha_max.name = "fecha_max"

# 8. Exportación
nds_out = xr.Dataset({
    "t2m_max_mensual_c": da_monthly_max,
    "fecha_max": da_fecha_max
})

ds_out.to_netcdf(outfile)

```

6.2 Comentario metodológico

El punto más delicado del procesamiento fue la dimensión temporal. En algunos archivos de Copernicus, el tiempo no aparece directamente como `time`, sino como `valid_time` o `forecast_time`. Por esta razón, el script incorpora una etapa de normalización de nombres antes de aplicar operaciones como `resample()` o extraer rangos temporales.

7 Verificación del cubo procesado

En esta sección se inspecciona el archivo de salida generado por el pipeline. La verificación permite comprobar que el producto derivado contiene las variables esperadas y mantiene una estructura espacial y temporal consistente.

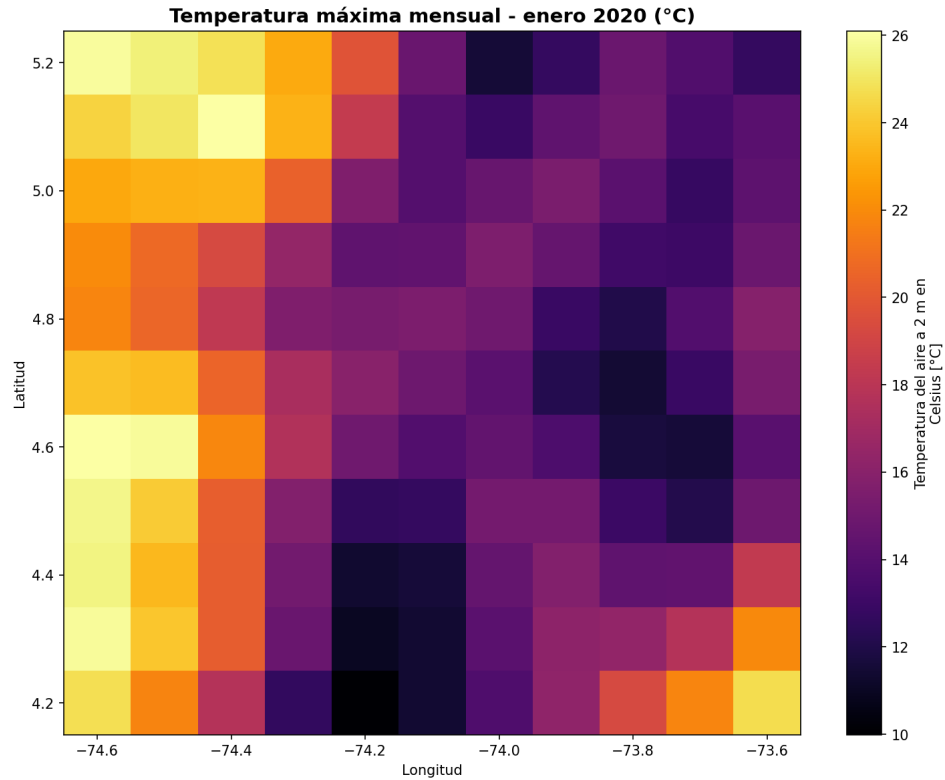
```
from pathlib import Path
import xarray as xr

data_dir = Path("./data_heavy")

ds_out = xr.open_dataset(data_dir / "mi_zona_procesada.nc", engine="netcdf4")
print(dict(ds_out.dims))
print(list(ds_out.data_vars))
print(ds_out)
```

8 Resultados gráficos

8.1 Figura 1. Mapa del máximo mensual



8.2 Figura 2. Serie diaria promedio del área de estudio

La segunda figura corresponde a la serie temporal diaria promedio de temperatura del área de estudio durante enero de 2020. Si el gráfico fue generado en el entorno local, puede insertarse en esta sección como imagen exportada. En caso contrario, la gráfica puede dejarse como resultado descrito en texto dentro del documento final.

9 Resultados principales

El procesamiento produjo un archivo NetCDF derivado con dos variables: una capa de máximos mensuales de temperatura y una segunda variable con la fecha de ocurrencia de ese máximo para cada píxel. La temperatura fue transformada de Kelvin a Celsius antes del remuestreo,

y el cubo horario fue reducido a una estructura diaria, lo que disminuyó sustancialmente el volumen de datos activo durante el análisis.

La figura incluida permite verificar la distribución espacial del máximo mensual dentro del área de estudio. Además, la inspección del archivo de salida confirma que el remuestreo temporal fue aplicado correctamente y que el eje de tiempo quedó reconocido como fecha real.

10 Respuestas analíticas

10.1 Sobre evaluación perezosa

Si se omitiera el remuestreo temporal y se intentara reproyectar directamente el cubo horario completo a un sistema local, el costo computacional crecería de manera importante. La reproyección no consiste únicamente en cambiar un metadato de referencia espacial, sino en recalcular los valores sobre una malla nueva. Cuando el cubo todavía conserva toda la resolución horaria, ese recálculo debe hacerse sobre una cantidad mucho mayor de datos.

Por eso, el warping debe dejarse para etapas tardías del pipeline. Primero conviene filtrar, transformar unidades y colapsar la dimensión temporal. Solo después de reducir el tamaño lógico del cubo tiene sentido reproyectar. Ese orden disminuye el uso de memoria RAM y evita reprocesar información que luego iba a resumirse.

10.2 Sobre convenciones CF y tiempo

En esta solución con Python y `xarray`, la dimensión temporal fue interpretada de forma automática una vez que el archivo fue abierto. Sin embargo, sí fue necesario normalizar el nombre de la coordenada temporal, porque el archivo podía venir con `valid_time` en lugar de `time`. Una vez hecho ese ajuste, fue posible utilizar `resample()` y trabajar con fechas reales sin reconstrucción manual adicional.

Esto muestra que la interpretación de las convenciones CF en Python es bastante fluida, aunque no siempre uniforme en los nombres con que aparecen las dimensiones o coordenadas del archivo.

10.3 Discusión técnica

Desde el punto de vista operativo, el paso más sensible no fue la conversión Kelvin–Celsius, sino la compatibilidad del backend de lectura y la normalización de nombres de coordenadas. El archivo pudo abrirse correctamente usando `engine="netcdf4"`, y a partir de ahí fue posible continuar con el pipeline sin necesidad de cargar el cubo completo en memoria de forma inmediata.

Otro aspecto importante fue la separación entre el script de descarga y el script de procesamiento. Esta decisión mejora la reproducibilidad del proyecto, porque permite validar la adquisición de datos y el tratamiento analítico como etapas distintas, manteniendo además una estructura de carpetas más clara.

11 Conclusión

El flujo desarrollado permitió pasar de un archivo NetCDF bruto descargado desde Copernicus CDS a un producto derivado más útil para análisis posteriores. La lectura perezosa ayudó a controlar el costo computacional, la transformación de unidades aseguró consistencia física, el remuestreo diario redujo el volumen activo del cubo y el cálculo del máximo mensual con su fecha de ocurrencia añadió una salida analítica relevante.

El resultado final quedó almacenado nuevamente en formato NetCDF, conservando la estructura multidimensional y dejando un insumo reutilizable para cartografía, modelación climática local o integración con flujos raster y vectoriales posteriores.

12 Anexo. Nota sobre el repositorio

El repositorio de entrega debe incluir el documento fuente `.qmd`, sus salidas HTML y PDF, y los scripts de descarga y procesamiento. Los archivos `.nc` no deben subirse a GitHub; por eso, la carpeta `data_heavy/` debe permanecer excluida mediante `.gitignore`.